

MTU
Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

ELX4 Assignment 1: Dijkstra Route Planner

By: Alan O'Connell

Lecturer: Donal O'Donovan

Module: Object Programming

Date: 22/10/2026

I acknowledge, by submitting this report, that I am aware of the provisions in MTU's Student Regulations which specifically cover cheating where any attempt is made to gain unfair advantage, whether in coursework, assessments or examinations.

I further acknowledge that I understand the Policy and associated Procedures together with the Supplementary Policy and Supplementary Procedure which have been approved for this period.

Contents

Introduction 2
1. Design Rationale 2
1.1 UML Class Diagram 2
1.2 Design Decision Log 2
1.2.1 How Dijkstra refers to its CollegeMap (D1) 2
1.2.2 Supplying the weights to other objects (CM8) 3
1.2.3 Copy constructor and assignment operator (CM7) 3
1.2.4 Overloaded operators 3
1.2.5 Error handling (CM2, CM5, D3) 3
1.2.6 const-correctness 4
1.2.7 2D storage layout (CM3) 4
1.2.8 Dijkstra working memory (D2) 4
1.2.9 Member data versus argument passing 4
1.2.10 Copying Dijkstra objects (D6) 4
1.3 Memory Lifecycle 5
1.4 Reference Code Defects 6
2. Unit Testing with Google Test 6
2.1 Test Strategy 6
2.2 Hand Working for Source Node 3 7
2.3 Traceability 8
2.4 Results 9
2.5 Defects Found During Development 10
3. AI Comparison Critique 10
3.1 Test-Based Comparison 10
3.2 Design Comparison 13
3.3 Constraint Compliance 13
3.4 Accuracy of the AI's Explanations 15
3.5 Test Quality 15
3.6 Reflection 15
Conclusion 15
References 16
Appendix A: Source Code 17
A.1 CollegeMap.h 17
A.2 CollegeMap.cpp 18

A.3 Dijkstra.h 23
A.4 Dijkstra.cpp 24
A.5 main.cpp (MapDemo) 28
A.6 test.cpp (MapTests) 30
Appendix B: AI Transcript 36
B.1 Prompt 1 36
B.2 Response 1 42
B.3 Prompt 2 (follow-up 1) 43
B.4 Response 2 43

Introduction

This report covers Assignment 1, a route planner built from two C++ classes. CollegeMap loads a map name, a node count and an edge-weight matrix from a text file such as 'MTU Map.txt' and keeps the weights in dynamically allocated memory. Dijkstra uses a CollegeMap to find the shortest distance and route from a chosen source node to every other node. Dijkstra aggregates CollegeMap, so the map exists on its own and can outlive any Dijkstra object that uses it.

A static library (MapLib) holds both classes and a console demo (MapDemo) and a Google Test project (MapTests) link against it. Section 1 explains the design, Section 2 covers the unit tests and Section 3 compares this solution with an AI-generated one. The full source code is in Appendix A and the AI transcript in Appendix B.

1. Design Rationale

1.1 UML Class Diagram

Figure 1 shows both classes with their member data and full signatures (− private, + public) [7]. Aggregation is a hollow diamond at the Dijkstra end with an open arrow to CollegeMap, since only Dijkstra knows about the other class. One map can serve any number of Dijkstra objects (0..*), each reaching it through `pMap`.

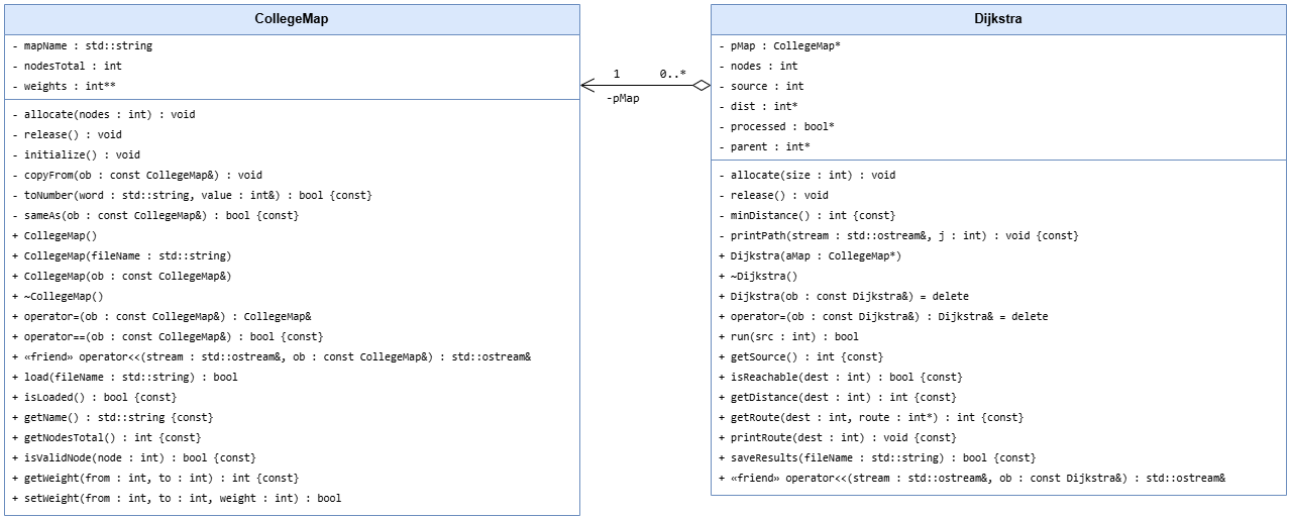


Figure 1: UML class diagram of CollegeMap and Dijkstra

1.2 Design Decision Log

1.2.1 How Dijkstra refers to its CollegeMap (D1)

Options: (a) a copy of the map, (b) a reference member, (c) a pointer.

Choice: a private pointer, `CollegeMap *pMap`, passed to the constructor as `&map`.

Reason: A stored copy is composition: it is created and destroyed with the Dijkstra object and never sees changes to the original, so option (a) is not aggregation. A reference member cannot be re-seated and blocks assignment.

Map destroyed first: If the map goes out of scope before the Dijkstra object, `pMap` dangles. Dijkstra.h only notes that the map is not owned. Preventing this would need shared ownership through smart pointers, which Fixed Constraint 3 rules out. `DijkstraSeesMapChanges` and `MapSurvivesDijkstraDestruction` show that Dijkstra uses the original map and that the map outlives it.

1.2.2 Supplying the weights to other objects (CM8)

Options: (a) return the internal `int**`, (b) return an independent copy of the matrix, (c) element access only.

Choice: element access through `getWeight(from, to)`.

Reason: Returning the internal pointer would let the calling code skip the index checks or delete the array itself, which would then be deleted twice. Returning a copy is safe, but it copies the whole matrix on every call and leaves the caller to free it. Element access hands out no memory, so only CollegeMap looks after the matrix, and `getWeight()` checks both indices and returns −1 for a bad one.

1.2.3 Copy constructor and assignment operator (CM7)

Signatures: `CollegeMap(const CollegeMap &ob)` and `CollegeMap &operator=(const CollegeMap &ob)`.

Reason: The copy constructor must take a reference, since a by value parameter would need a copy to make the copy. `operator=` could take its parameter by value, but `copyFrom()` already makes the copy, so a reference avoids a second one. `const` lets the function accept temporary and const maps, and stops it from changing the map being copied. Returning `*this` by reference lets assignments chain (`a = b = c`).

Self-assignment: `copyFrom()` builds the new copy in a temporary array before freeing the old one, so `a = a` works without a `this == &ob` check. The cost is that `a = a` still copies the whole matrix. `SelfAssignmentKeepsData` tests this.

Avoiding duplicated logic: The copy constructor and `operator=` both call the private helper `copyFrom()`, so the copying code is only written once. The copy constructor calls `initialize()` first to set the members to empty, so the `release()` inside `copyFrom()` has nothing to free and is safe.

1.2.4 Overloaded operators

Choice: =, == and a friend <<. == calls `sameAs()`, a private helper that compares the name, size and every weight, so a copy test needs one line (`EXPECT_TRUE(copy == original)`). << is a friend because the stream is the left-hand operand. It takes any `ostream`, so the same operator also writes the results file.

Considered and rejected: `operator[]`, because it would expose the raw rows and bypass the checks in `setWeight()`. + and - have no clear meaning for a map.

1.2.5 Error handling (CM2, CM5, D3)

Options: (a) report errors through return values (`false` or −1), (b) throw exceptions.

Choice: return values. `load()` returns `false` and prints a message if the file is missing, the node count is bad, there are too few or non-numeric weights, a weight is negative or the diagonal is not zero. It checks each value with `isdigit()` in `toNumber()` before `atoi()` converts it. After a failed load the map is left empty ("None", 0 nodes, no array) and an empty map can still be displayed, copied, assigned and destroyed safely.

Non-symmetric maps: The supplied 'MTU Map.txt' is not symmetric at (3,5) and (3,6), so a non-symmetric map only gives a warning. Rejecting it would stop the supplied map from loading. `getWeight(u, v)` treats each entry as one-way, so those two edges only work in one direction, while `setWeight()` always sets both directions.

Invalid indices and sources: A bad index makes `getWeight()` return −1, which can never be a real weight, and makes `setWeight()` return `false` without changing anything. A bad source node makes `run()` return `false` and clears the old source, so old results cannot be read by mistake. An unreachable node gives −1 from `getDistance()` and prints as "unreachable" instead of 2147483647 (D4).

1.2.6 const-correctness

Parameters: Every function that takes a CollegeMap or Dijkstra object takes it as a const reference: the copy constructor, `operator=`, `==`, `copyFrom()`, `sameAs()` and both `<<` operators. Passing a map by value would copy the whole weight matrix on every call and `const` stops the function from changing it. The Dijkstra constructor is different. It takes a `CollegeMap*` because it stores the map's address.

Methods: Every method that only reads data is `const` [6]. A function holding a map as a `const` reference can only call `const` methods, so without them it could not ask the map for its size. The compiler also stops a `const` method from changing the object by mistake. `setWeight()`, `load()` and `run()` change the object, so they are not `const`. Figure 1 shows the full list of `const` methods.

1.2.7 2D storage layout (CM3)

Options: (a) an array of row pointers (`int**`), each row allocated separately, (b) one contiguous n × n block indexed as `row * n + col`.

Choice: the array of row pointers.

Reason: It keeps the `weights[row][col]` syntax of the reference code and the example in the brief. The downside is that each row is created separately, so it takes more steps to set up than one single block. With only 9 nodes, this makes no real difference.

1.2.8 Dijkstra working memory (D2)

Options: (a) allocate `dist`, `processed` and `parent` once in the constructor, (b) allocate them on every run.

Choice: `run()` frees the old arrays and allocates new ones sized to `pMap->getNodesTotal()`.

Reason: The calling code can reload the map with a different node count after the Dijkstra object exists and arrays sized in the constructor would then overflow. Weight changes apply on the next run because weights are read through the pointer, but earlier results are stale until `run()` is called again. `DijkstraSeesMapChanges` tests both cases.

1.2.9 Member data versus argument passing

The reference shortest path code [1] passes data into every function, with the size fixed by the global V. In the class, the map pointer, node count, source and working arrays are member data that is available to every method, so `minDistance()` takes no arguments and the run-time node count replaces V. `printPath()` is a private helper that calls itself to print a route and it writes to whichever stream it is given. `printRoute()` and the friend `<<` both use it, so one piece of code prints routes to both the screen and the results file. The friend `<<` replaces the reference code's `printSolution()`.

1.2.10 Copying Dijkstra objects (D6)

Options: (a) allow the default copy, (b) write a deep copy, (c) block copying.

Choice: block copying, by declaring the copy constructor and `operator=` as `= delete` [2].

Reason: A default copy would share the working arrays and both objects would `delete[]` them. A deep copy adds little, since `run()` regenerates the results. `= delete` keeps the functions in the class declaration, but the compiler rejects any code that calls them. The two declarations in Dijkstra.h are:

```
Dijkstra(const Dijkstra& ob) = delete;
Dijkstra& operator=(const Dijkstra& ob) = delete;
```

As a result, `Dijkstra b = a;` fails with error C2280, "attempting to reference a deleted function" (Figure 2), while leaving them undeclared would allow a silent shallow copy.

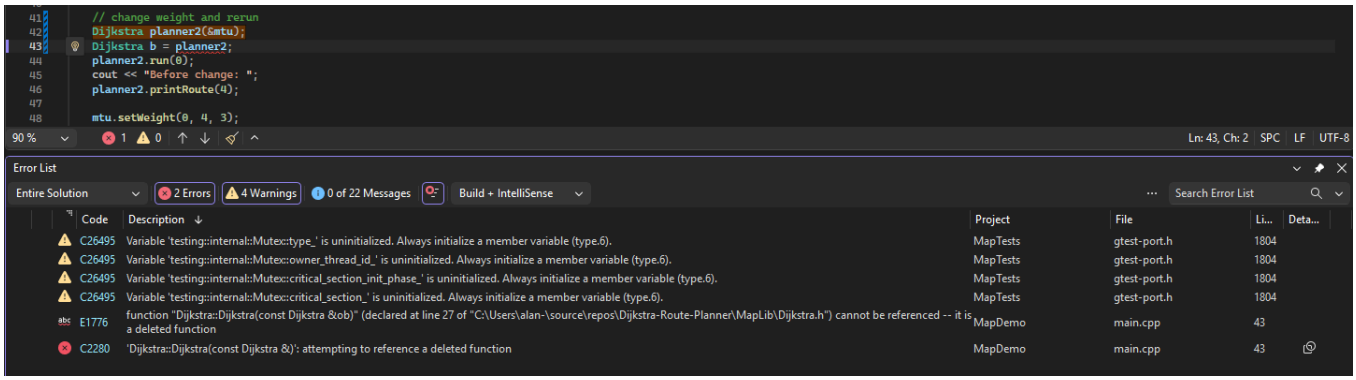


Figure 2: Error C2280 when copying a Dijkstra object

1.3 Memory Lifecycle

Table 1 matches every new in the code to its `delete`.

Allocation	Where	Released by	Release called from
weights = new int*[nodesTotal] (temp in copyFrom())	allocate(), copyFrom()	delete[] weights in release()	Destructor, start and error paths of load(), copyFrom() after copying
weights[row] = new int[nodesTotal] per row (temp[row] in copyFrom())	allocate(), copyFrom()	delete[] weights[row] loop in release()	As above, rows first
dist = new int[nodes]	Dijkstra::allocate()	delete[] dist in release()	Destructor, start of run()
processed = new bool[nodes]	Dijkstra::allocate()	delete[] processed in release()	Destructor, start of run()
parent = new int[nodes]	Dijkstra::allocate()	delete[] parent in release()	Destructor, start of run()
planner = new Dijkstra(&mtu)	main() (MapDemo), also MapSurvivesDijkstraDestruction	delete planner	Same function, after use

Table 1: new / delete pairs

Both `release()` functions check before deleting and reset their pointers to 0, so repeated calls are harmless. Every block of memory is freed exactly once on every path through the program, including when a file fails to load. The one exception is if new throws part-way through building a matrix in `copyFrom()` or `allocate()`: the rows already created are not freed (Section 3.2).

Leaks were checked with the Visual Studio Diagnostic Tools window [3], using Debug-build heap snapshots while MapDemo ran from source node 3 (Figure 3). Snapshots 1 and 2, taken before `new Dijkstra(&mtu)`, show 24 allocations. Snapshot 3, taken just before `delete planner`, shows 28 (+4), the Dijkstra object and its `dist`, `processed` and `parent` arrays. Snapshot 4, taken after

`delete planner`, is back to 24 (−4), so the Dijkstra object frees everything it allocates and leaves the map's memory alone. The snapshots cannot show the CollegeMap objects being freed, because they stay in scope until main() ends. Instead, their memory is freed by `release()`, which the destructor calls, and `ReloadReplacesOldMap` checks that loading a new map frees the old one first.

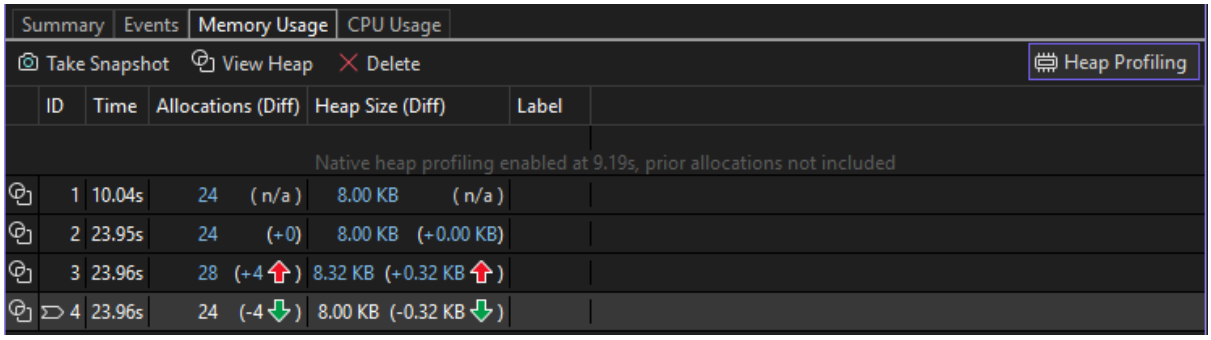


Figure 3: Diagnostic Tools heap snapshots around the Dijkstra object's lifetime

1.4 Reference Code Defects

The reference code in the brief's Appendix 1 has three defects. All three stay hidden because it only ever runs from node 0 on a fully connected map.

The initialisation loop sets `parent[0] = -1` instead of `parent[i] = -1`, so every other entry is left holding a random value. From node 0 this does not matter, because `parent[0]` is the source's own entry. From any other source, `parent[src]` holds a random value and `printPath()` follows it into undefined behaviour. `run()` now sets every `parent[i]` to −1 and `RouteFromNodeThreeToSeven` tests it.

`minDistance()` compares with <= against `INT_MAX`. Once every reachable node is processed, it can still pick an unreachable one and `dist[u] + w` then overflows to a negative distance. The sample map is fully connected, so this never happens. `minDistance()` now uses < and returns −1 when no reachable node is left, which ends the loop in `run()`. `UnreachableNodeReported` tests it.

`printSolution()` always prints source 0. The reference code only ever runs from node 0, so the output still looks correct. The friend << now prints the real source and `PrintAllShowsRealSource` tests it.

2. Unit Testing with Google Test

2.1 Test Strategy

MapTests is a separate Google Test project [4] linked to MapLib, so its 20 tests call the classes directly rather than going through the demo. Each test name says what it checks and a comment above each test lists the behaviours it covers (CM1 to CM9, D1 to D7). ASSERT_* is used where later checks depend on an earlier result, so the test stops if that result is wrong. Everywhere else EXPECT_* is used, so the remaining checks still run after a failure.

The tests create their own map files instead of relying on 'MTU Map.txt'. Following the brief's Appendix 4, the three fixtures in Table 2 write their map file in `SetUp()` and delete it in `TearDown()`, so no test depends on another.

Fixture	Map
TinyMapTest	tiny.txt: 3-node map from the brief (0-1 weight 5, 1-2 weight 2)
MtuMapTest	mtu.txt: Appendix 3 data, written by the test
SplitMapTest	split.txt: two unconnected parts, 0-1 and 2-3

Table 2: Test fixtures

Other tests create and remove their own one-off maps: four malformed files, a one-node map, and 2 and 4 node maps for the reload tests. Output from both << operators and `saveResults()` is written to a file and read back word by word. `RouteFromNodeThreeToSeven` also calls `printRoute()` with a valid and an invalid destination to check that neither call crashes. The printed text is not checked.

The expected values were derived without running the program. The 3- and 4-node maps are small enough to check by hand, the MTU source-0 values come from the brief's sample output and the source-3 values were worked out by hand (Section 2.2).

D6 has no runtime test. Copying a Dijkstra object is a compile error, so a test that tried it would stop the whole test project building (Figure 2).

2.2 Hand Working for Source Node 3

Table 3 traces the algorithm [5] on the MTU map from node 3, using the weights as stored (row = from, column = to) and breaking ties by the lower node number. Table 4 gives the resulting distances and routes.

Step	Pick (dist)	Updates
1	3 (0)	2: 0 + 7 = 7, 4: 0 + 9 = 9, 5: 0 + 14 = 14
2	2 (7)	1: 7 + 8 = 15, 5: 7 + 4 = 11 (was 14), 8: 7 + 2 = 9
3	4 (9)	5: 9 + 10 = 19, no change
4	8 (9)	6: 9 + 6 = 15, 7: 9 + 7 = 16
5	5 (11)	6: 11 + 2 = 13 (was 15)
6	6 (13)	7: 13 + 1 = 14 (was 16)
7	7 (14)	0: 14 + 8 = 22, 1: 14 + 11 = 25, no change
8	1 (15)	0: 15 + 4 = 19 (was 22)
9	0 (19)	All nodes processed

Table 3: Hand working from node 3

Destination	Distance	Route
0	19	3 → 2 → 1 → 0
1	15	3 → 2 → 1
2	7	3 → 2
4	9	3 → 4
5	11	3 → 2 → 5
6	13	3 → 2 → 5 → 6
7	14	3 → 2 → 5 → 6 → 7
8	9	3 → 2 → 8

Table 4: Expected results from node 3

2.3 Traceability

Behaviour	Test name(s)	Notes
CM1	EmptyMapIsSafeToUse	Empty map safe to copy, assign and display
CM2	MissingAndMalformedFilesFailToLoad, DistancesFromNodeZeroMatchBrief	Missing file and four malformed files, asymmetric MTU map loads and runs
CM3	LoadsNameNodesAndWeights	Weights checked after loading
CM4	LoadsNameNodesAndWeights, DisplayShowsNameNodesAndMatrix	Name and node count
CM5	InvalidIndicesAndWeightsRejected, SetWeightChangesBothDirections	Indices −1 and n, rejected value leaves map unchanged
CM6	DisplayShowsNameNodesAndMatrix, EmptyMapIsSafeToUse	Output read back word by word
CM7	CopyIsIndependent, AssignmentIsIndependent, SelfAssignmentKeepsData	Independence checked both ways
CM8	DistancesFromNodeZeroMatchBrief, RerunFromNodeThree, DijkstraSeesMapChanges	Dijkstra reads weights via getWeight()
CM9	ReloadReplacesOldMap	Leak check: Section 1.3
==	CopyIsIndependent, AssignmentIsIndependent	Equal and unequal maps
D1	DijkstraSeesMapChanges, MapSurvivesDijkstraDestruction	Uses the original map, not a copy
D2	DijkstraSeesMapChanges	Map reloaded at a larger size
D3	OneNodeMapRuns, DistancesFromNodeZeroMatchBrief, RerunFromNodeThree, InvalidSourceAndEmptyMapRejected	Reruns, bad sources and empty map
D4	RouteFromNodeThreeToSeven, UnreachableNodeReported, PrintAllShowsRealSource	Unreachable gives −1 and no route
D5	SaveResultsWritesMapAndRoutes	Saved file checked
D6	None	Compile error in Figure 2
D7	MapSurvivesDijkstraDestruction	Release check: Section 1.3
Ref. defect 1	RouteFromNodeThreeToSeven	parent[] initialisation
Ref. defect 2	UnreachableNodeReported	<= in minDistance()
Ref. defect 3	PrintAllShowsRealSource	Hard-coded source 0

Table 5: Traceability

2.4 Results

All 20 tests pass (Figure 4).

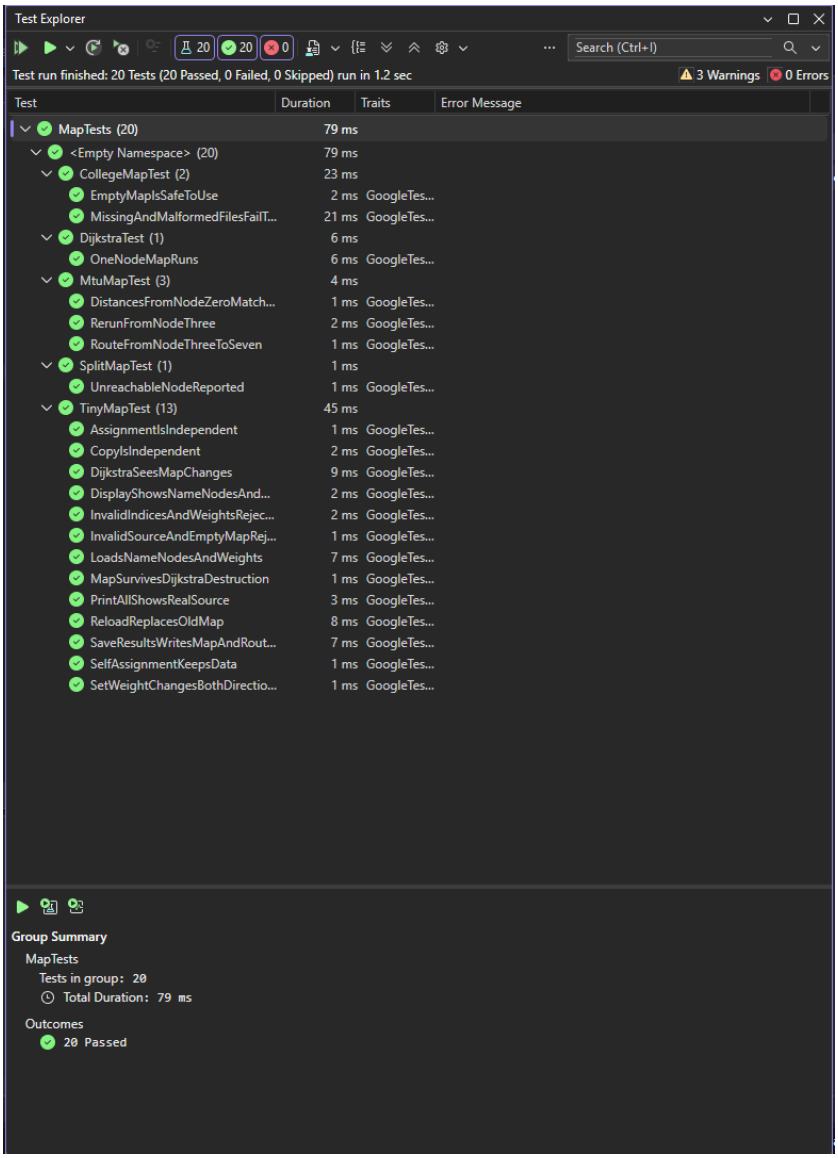


Figure 4: Test Explorer, 20 tests passed

Each reference defect has a regression test: `UnreachableNodeReported` targets the `<=` in `minDistance()`, `PrintAllShowsRealSource` targets the hard-coded source 0 in `<<`, and `RouteFromNodeThreeToSeven` targets `parent[0] = -1`. To check that each test catches its defect, each faulty line was swapped back into Dijkstra.cpp one at a time, the test suite was rerun, and the correct line was then restored. Changing `parent[i] = -1` back to `parent[0] = -1` made `RouteFromNodeThreeToSeven` crash and so did `PrintAllShowsRealSource`, since both follow a route from a non-zero source. Changing `<` to `<=` in `minDistance()` made only `UnreachableNodeReported` fail and hard-coding source 0 in `<<` made only `PrintAllShowsRealSource` fail. With all three lines restored the 20 tests passed.

2.5 Defects Found During Development

Most problems during development were small mistakes or oversights, but one was a real flaw in the classes. The first `load()` read the node count straight into an `int`, so it took "9.0" as 9 and only rejected the file at the first weight, with a misleading "too few or non-numeric weights" message. `toNumber()` now checks each character of the count with `isdigit()`, so `load()` rejects the file at the count.

MapTests initially would not build because a stale MapTests.exe from an earlier test run was still running and locking the file. Ending the process in Task Manager fixed it.

3. AI Comparison Critique

The AI was given the brief from "Assignment Overview" to "Sample Output" with Appendices 1 and 3 from the brief. It listed the reference defects and asked what was wanted. One follow-up prompt then produced a zip with both classes, a demo and 65 Google Tests, but no Visual Studio project files.

The solution then had to be assembled by hand: a static library, a demo and two test projects, each needing include paths, a library reference and precompiled header settings. Getting it to compile took several attempts and errors.

AI assistant used: Claude Opus 5.5, in a new chat with no earlier context.

Follow-up prompts used: 1.

3.1 Test-Based Comparison

The 20 MapTests tests were ported without changing what each one checks (AISolution/PortedTests/test.cpp). The AI's interface forced the changes: renamed methods, a Dijkstra constructor that takes a reference instead of a pointer, errors reported by exceptions or a `LoadStatus` instead of `false` or '−1' and routes returned as strings rather than `int` arrays.

On the supplied Appendix 3 map, 13 of the 20 tests pass on the AI code and all the AI's 65 tests pass (Table 6, Figure 5).

Test name	Own code	AI code	Change needed / reason for failure
EmptyMapIsSafeToUse	Pass	Fail	Renames, empty name “” not “None”, layout: extra “(no weights)”
MissingAndMalformedFilesFailToLoad	Pass	Pass	LoadStatus, not bool
LoadsNameNodesAndWeights	Pass	Pass	Renames
InvalidIndicesAndWeightsRejected	Pass	Pass	Throws instead of −1 / false
SetWeightChangesBothDirections	Pass	Pass	void setWeight()
DisplayShowsNameNodesAndMatrix	Pass	Fail	Layout: row labels, extra line
CopyIsIndependent	Pass	Pass	Renames
AssignmentIsIndependent	Pass	Pass	Renames
SelfAssignmentKeepsData	Pass	Pass	Renames
ReloadReplacesOldMap	Pass	Pass	LoadStatus, not bool
OneNodeMapRuns	Pass	Pass	void run(), distanceTo()
DistancesFromNodeZeroMatchBrief	Pass	Fail	MTU map rejected, run() throws
RerunFromNodeThree	Pass	Fail	MTU map rejected, run() throws
RouteFromNodeThreeToSeven	Pass	Fail	MTU map rejected, string route
InvalidSourceAndEmptyMapRejected	Pass	Pass	run() throws
UnreachableNodeReported	Pass	Pass	Route string “unreachable”
PrintAllShowsRealSource	Pass	Fail	Layout: extra header line
SaveResultsWritesMapAndRoutes	Pass	Fail	No weight matrix in file
DijkstraSeesMapChanges	Pass	Pass	void run(), LoadStatus
MapSurvivesDijkstraDestruction	Pass	Pass	Reference constructor

Table 6: 20 MapTests tests against both solutions

Three failures come from the AI rejecting the non-symmetric MTU file. Every `MtuMapTest` test fails because `run()` throws on the rejected map and the AI’s demo shows the same rejection (Figure 7).

Three failures are from output layout. The display tests compare output word by word and the AI prints extra text: “Nodes: 0 (no weights)” on the empty map, “0:” row labels and a “(0 = no direct edge)” line on the matrix and a “Map: Tiny_Map Nodes: 3” line before its route table. The values match wherever they are visible, so the failures come from word-by-word checks written against this solution’s output format.

`SaveResultsWritesMapAndRoutes` fails because the AI’s `saveResults()` writes the same text as `writeResults()`, which its own test checks, so the file holds the “Map: Tiny_Map Nodes: 3” line and the route table but not the weight matrix this test expects.

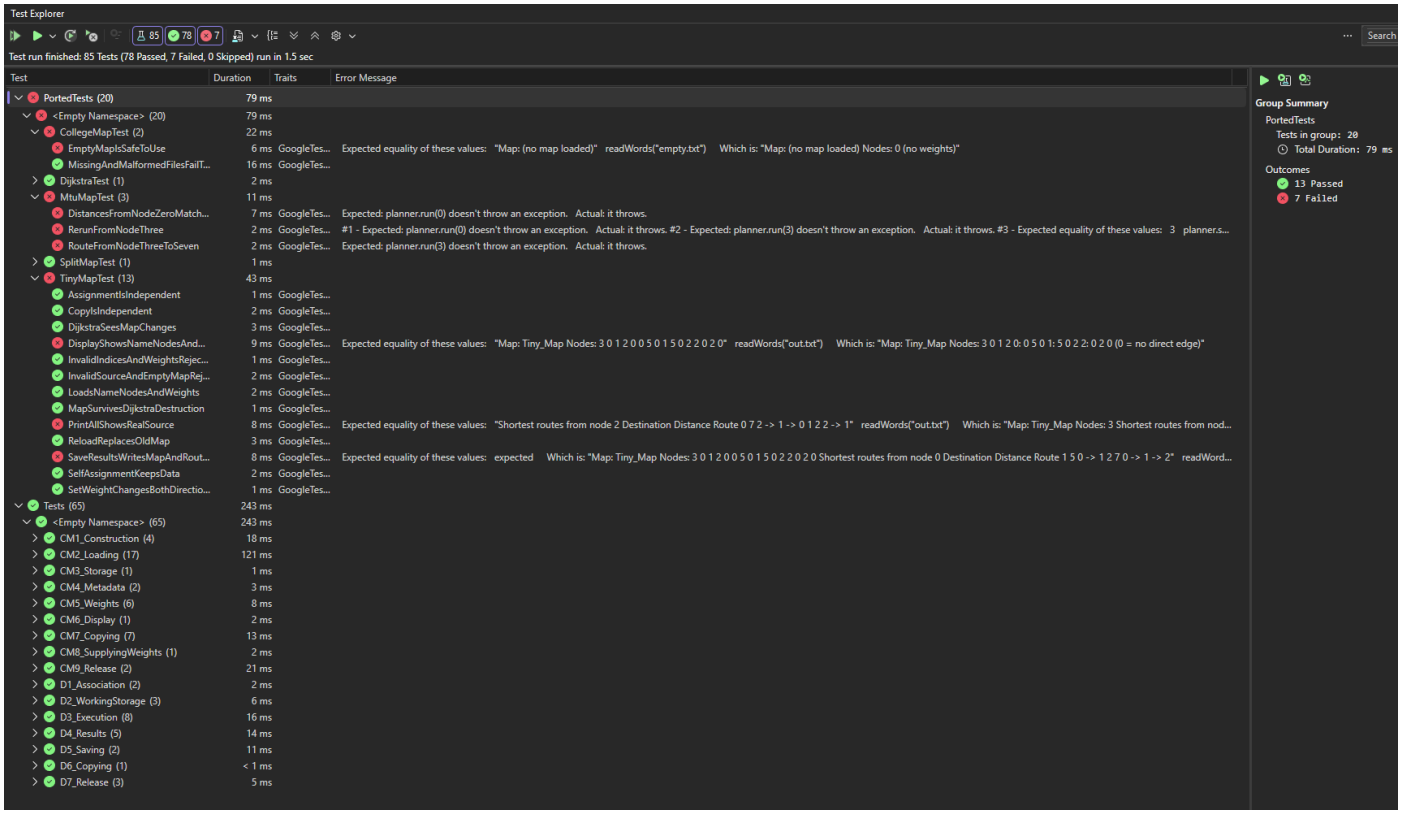


Figure 5: Test Explorer for the AI solution

3.2 Design Comparison

Decision	This solution	AI solution	Assessment and evidence
Q1 Map association (D1)	CollegeMap* passed to the constructor	const CollegeMap& member, deleted rvalue constructor	Both aggregation. The AI's deleted constructor blocks temporary maps. The AI solution seems better here.
Q2 Supplying weights (CM8)	getWeight(), −1 for a bad index	weight() and operator()(r, c) by value, throw on a bad index	Same idea: no memory handed out. Equal.
Q3 Copying (CM7)	copyFrom() into a temp	Copy-and-swap [8] plus move constructor	AI leaves no leak if new[] throws part-way, while this leaks the copied rows. The AI solution seems better here.
Q4 Operators	=, ==, <<	=, ==, !=, << and read-only operator()	!= and operator() return by value, so tests get shorter without exposing a row. The tests cover the same checks with ==. AI better by a small margin.
Q5 Error handling	bool returns, failed load empties map, asymmetry warns	LoadStatus and exceptions, failed load keeps old map, asymmetry is an error	AI reports more detail, but rejecting asymmetry blocks the brief's map, failing three ported tests from Part A. Mixed.
Q6 const-correctness	const getters and references, strings by value	Same, plus const std::string& returns and noexcept	const std::string& returns save one string copy per call, which makes no major difference for 9 nodes. AI better by a small margin.
Q7 2D storage (CM3)	Row pointers (int**)	One contiguous block	One new[]/delete[] simplifies copying and clean-up, where copyFrom() leaks. AI does a better job.
Q8 Working memory (D2)	Reallocated every run()	Reallocated every run(), revision counter marks stale results	AI refuses stale queries through its revision counter, while this solution returns them. AI is better.
Q9 Member data	Map, size, source and arrays as members	Same. Its minDistance() equivalent, closestUnprocessed(), is a file-local helper	Both keep the same state as members. A file-local minDistance() and a private one look the same to callers. Equal.
D6 Dijkstra copying	= delete	= delete	Same idea, but the AI enforces it with a static_assert test [10], while this solution relies on the error in Figure 2. AI did it better.

Table 7: Design comparison

3.3 Constraint Compliance

The AI keeps every Fixed Constraint that could be checked: separate .h and .cpp files, no inheritance, `new[]` and `delete[]` for the matrix and working arrays, `std::string` only for text and aggregation through a const reference that is never freed. Its tests replace the global operator `new[]` and `delete[]` to count allocations, including forced failures.

The AI said it never built its code with MSVC, so it was built in Visual Studio 2026 at /W3 for this comparison. Both solutions build with no warnings, so the AI also meets Fixed Constraint 6. Its demo rejects 'MTU Map.txt' and loads its own edited copy, moving the 14 from (6,3) to (5,3) (Figure 7), while this solution's demo loads the supplied file with two asymmetry warnings (Figure 6).

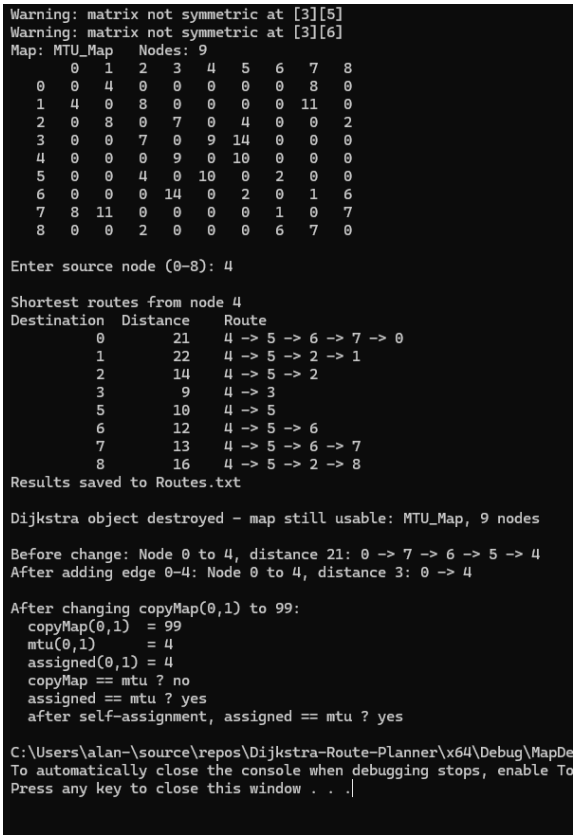


Figure 6: Part A solution output



Figure 7: AI Solution Output

3.4 Accuracy of the AI's Explanations

The claims that could be checked against the code held up. All 65 of the AI's tests pass (Figure 5), the solution builds without warnings at /W3, and its demo rejects 'MTU Map.txt' for the reason the AI gave: row 3 has 14 in column 5, row 5 has 0 in column 3, so `load()` returns `NotSymmetric` (Figure 7).

Its account of what it delivered was less accurate. The AI said the zip contained a demo project and a Google Test project, but it held only the .h and .cpp files, the test source and a README in separate folders per section, with no Visual Studio project files. The static library, the demo and both test projects each needed to be created in Visual Studio by hand, with their own include paths, library reference and precompiled header settings.

The AI made a good point. The Part A solution treated the map as asymmetric because that is how it was supplied and the likely error only became clear after the AI changed it: one 14 is a row out of place, at (6,3) instead of (5,3), and moving it makes the matrix symmetric. There is no definite answer on who's right, as CM2 leaves symmetry as a design choice and the brief's Appendix 1 code uses the same matrix. The AI replaced the supplied file while the Part A solution loads it as given with a warning, and both produce the brief's node-0 results. The Part A code was not changed afterwards, as changing it would have been influenced by the AI.

3.5 Test Quality

The AI's 65 tests build traceability into their names (for example `CM7_Copying`) and their expected values are independent: node-0 values from the brief, a test that checks all 81 distances with a second algorithm and a check that every route is a real path of the reported length. Its allocation counting checks Constraint 5, including failure paths, which is stronger than the Diagnostic Tools snapshots used in Part A and `static_assert` gives D6 a real test.

It uses no `TEST_F` fixtures, although the brief asks for them, and it only tests the supplied file for rejection. Its output checks search for text fragments, so `TableListsEveryOtherNode` checks two of the eight routes.

3.6 Reflection

The comparison points to three changes. `copyFrom()` needs to avoid leaking when an allocation fails part-way, either by switching to a contiguous block or by freeing the rows already copied. The AI's stale-results check and allocation-counting tests are also worth considering.

The AI's reading of the asymmetry as a misplaced value is likely correct and was missed in Part A. Rejecting the file is still not worth adopting, since it stops the demo running on the supplied map and fails all three MTU tests. The `bool` and −1 returns stay, although exceptions would suit a larger program.

Conclusion

Both classes meet the required behaviours within the Fixed Constraints. The matrix and working arrays use raw `new[]` and `delete[]`, each block is freed once on every normal path, and Dijkstra aggregates CollegeMap through a pointer it never deletes. The 20 Google Tests trace every behaviour except D6, which the compiler enforces, and each reference code defect has a regression test.

The same tests passed 13 of 20 on the AI's solution, with no failure pointing to a fault in its algorithm. The AI's design is stronger in places: copy-and-swap avoids the leak in `copyFrom()`, the revision counter catches stale results, and the AI spotted the likely error in the supplied map.

References

Most of the extra reading was on Microsoft Learn, having used it for across a few languages. Only the pages used are listed, leaving out the ones clicked through to find them.

[1] 'Printing Paths in Dijkstra's Shortest Path Algorithm', GeeksforGeeks. Accessed: Oct. 19, 2026. [Online]. Available: https://www.geeksforgeeks.org/dsa/printing-paths-dijkstras-shortest-path-algorithm/

[2] Microsoft, 'Explicitly Defaulted and Deleted Functions'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/cpp/explicitly-defaulted-and-deleted-functions?view=msvc-170

[3] Microsoft, 'Measure memory usage in your apps - Visual Studio (Windows)'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/visualstudio/profiling/memory-usage?view=visualstudio

[4] Microsoft, 'Create and run unit tests with Google Test for C++ - Visual Studio (Windows)'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/visualstudio/test/how-to-use-google-test-for-cpp?view=visualstudio

[5] 'Find Shortest Paths from Source to all Vertices using Dijkstra's Algorithm', GeeksforGeeks. Accessed: Oct. 19, 2026. [Online]. Available: https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/

[6] Microsoft, 'const (C++)'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/cpp/const-cpp?view=msvc-170

[7] 'UML Class Diagram', GeeksforGeeks. Accessed: Oct. 19, 2026. [Online]. Available: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/

[8] 'Copy-and-Swap Idiom in C++', GeeksforGeeks. Accessed: Oct. 19, 2026. [Online]. Available: https://www.geeksforgeeks.org/cpp/copy-swap-idiom-c/

[9] Microsoft, 'Move Constructors and Move Assignment Operators (C++)'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/cpp/move-constructors-and-move-assignment-operators-cpp?view=msvc-170

[10] Microsoft, 'static_assert'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/cpp/static-assert?view=msvc-170

[11] Google, 'GoogleTest Primer', GitHub. Accessed: Oct. 19, 2026. [Online]. Available: https://github.com/google/googletest/blob/main/docs/primer.md

[12] Microsoft, 'remove, _wremove'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/remove-wremove?view=msvc-170

[13] Microsoft, 'atoi, _atoi_l, _wtoi, _wtoi_l'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/atoi-atoi-l-wtoi-wtoi-l?view=msvc-170

[14] Microsoft, 'isdigit, iswdigit, _isdigit_l, _iswdigit_l'. Accessed: Oct. 19, 2026. [Online]. Available: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/isdigit-iswdigit-isdigit-l-iswdigit-l?view=msvc-170

Appendix A: Source Code

These are the final files for MapLib (A.1 to A.4), MapDemo (A.5) and MapTests (A.6).

A.1 CollegeMap.h

```
#ifndef COLLEGEMAP_H
#define COLLEGEMAP_H

#include <iostream>
#include <string>

class CollegeMap
{
    std::string mapName;
    int nodesTotal;     // 0 = no map
    int** weights;      // 2D weight array

    void allocate(int nodes);
    void release();
    void initialize();
    void copyFrom(const CollegeMap& ob);
    bool toNumber(std::string word, int& value) const;
    bool sameAs(const CollegeMap& ob) const;

public:
    CollegeMap();
    CollegeMap(std::string fileName);
    CollegeMap(const CollegeMap& ob);   // copy constructor
    ~CollegeMap();

    CollegeMap& operator=(const CollegeMap& ob);
    bool operator==(const CollegeMap& ob) const;
    friend std::ostream& operator<<(std::ostream& stream, const CollegeMap& ob);

    bool load(std::string fileName);

    bool isLoaded() const;
    std::string getName() const;
    int getNodesTotal() const;
    bool isValidNode(int node) const;

    int getWeight(int from, int to) const;
    bool setWeight(int from, int to, int weight);
};

#endif
```

A.2 CollegeMap.cpp

```
#include "CollegeMap.h"
#include <fstream>
#include <iomanip>
#include <cstdlib>
using namespace std;

// initialize member variables
void CollegeMap::initialize()
{
    mapName = "None";
    nodesTotal = 0;
    weights = 0;
}

// empty map
CollegeMap::CollegeMap()
{
    initialize();
}

// load from file
CollegeMap::CollegeMap(string fileName)
{
    initialize();
    load(fileName);
}

// copy constructor
CollegeMap::CollegeMap(const CollegeMap& ob)
{
    initialize();
    copyFrom(ob);
}

CollegeMap::~CollegeMap()
{
    release();
}

// make array, all zeros
void CollegeMap::allocate(int nodes)
{
    nodesTotal = nodes;
    weights = new int* [nodesTotal];
    if (!weights)
    {
        cout << "Allocation failed." << endl;
        exit(1);
    }
    for (int row = 0; row < nodesTotal; ++row)
    {
        weights[row] = new int[nodesTotal];
        if (!weights[row])
        {
            cout << "Allocation failed." << endl;
            exit(1);
        }
        for (int col = 0; col < nodesTotal; col++)
            weights[row][col] = 0;
    }
}

// free the array
void CollegeMap::release()
```

```
{
    if (weights)
    {
        for (int row = 0; row < nodesTotal; row++)
            delete[] weights[row];
        delete[] weights;
    }
    weights = 0;
    nodesTotal = 0;
    mapName = "None";
}

// deep copy
void CollegeMap::copyFrom(const CollegeMap& ob)
{
    int n = ob.nodesTotal;
    string name = ob.mapName;
    int** temp = 0;

    if (n > 0)
    {
        temp = new int* [n];
        if (!temp)
        {
            cout << "Allocation failed." << endl;
            exit(1);
        }
        for (int row = 0; row < n; ++row)
        {
            temp[row] = new int[n];
            if (!temp[row])
            {
                cout << "Allocation failed." << endl;
                exit(1);
            }
            for (int col = 0; col < n; col++)
                temp[row][col] = ob.weights[row][col];
        }
    }

    release();      // free old map
    weights = temp;
    nodesTotal = n;
    mapName = name;
}

// word to int, false if bad
bool CollegeMap::toNumber(string word, int& value) const
{
    int length = (int)word.length();
    if (length == 0)
        return false;

    int start = 0;
    if (word[0] == '-')
        start = 1;
    if (start == length)
        return false;

    for (int i = start; i < length; i++)
    {
        if (!isdigit(word[i]))
            return false;
    }
```

```
    value = atoi(word.c_str());
    return true;
}

// same name, size, weights
bool CollegeMap::sameAs(const CollegeMap& ob) const
{
    if (!(mapName == ob.mapName) || nodesTotal != ob.nodesTotal)
        return false;

    for (int row = 0; row < nodesTotal; row++)
        for (int col = 0; col < nodesTotal; col++)
            if (weights[row][col] != ob.weights[row][col])
                return false;

    return true;
}

CollegeMap& CollegeMap::operator=(const CollegeMap& ob)
{
    copyFrom(ob);
    return *this;
}

bool CollegeMap::operator==(const CollegeMap& ob) const
{
    return sameAs(ob);
}

// print name and matrix
ostream& operator<<(ostream& stream, const CollegeMap& ob)
{
    if (ob.nodesTotal == 0)
    {
        stream << "Map: (no map loaded)" << endl;
        return stream;
    }

    stream << "Map: " << ob.mapName << "   Nodes: " << ob.nodesTotal << endl;

    stream << "    ";
    for (int col = 0; col < ob.nodesTotal; col++)
        stream << setw(4) << col;
    stream << endl;

    for (int row = 0; row < ob.nodesTotal; row++)
    {
        stream << setw(4) << row;
        for (int col = 0; col < ob.nodesTotal; col++)
            stream << setw(4) << ob.weights[row][col];
        stream << endl;
    }
    return stream;
}

// read file, false if bad
bool CollegeMap::load(string fileName)
{
    release();

    ifstream fin(fileName);
    if (!fin)
    {
        cout << "Cannot open input file: " << fileName << endl;
        return false;
```

```
    }

    string name = "";
    string word = "";
    int nodes = 0;
    fin >> name >> word;
    if (!toNumber(word, nodes) || nodes <= 0)
    {
        cout << "Missing or invalid node count in: " << fileName << endl;
        return false;
    }

    allocate(nodes);

    for (int row = 0; row < nodesTotal; row++)
    {
        for (int col = 0; col < nodesTotal; col++)
        {
            word = "";
            fin >> word;
            if (!toNumber(word, weights[row][col]))
            {
                cout << "Too few or non-numeric weights in: " << fileName << endl;
                release();
                return false;
            }
            if (weights[row][col] < 0)
            {
                cout << "Negative weight at [" << row << "][" << col << "]" <<
endl;
                release();
                return false;
            }
        }
    }

    // diagonal 0
    for (int row = 0; row < nodesTotal; row++)
    {
        if (weights[row][row] != 0)
        {
            cout << "Non-zero diagonal at node " << row << endl;
            release();
            return false;
        }
        for (int col = row + 1; col < nodesTotal; col++)
        {
            if (weights[row][col] != weights[col][row])
                cout << "Warning: matrix not symmetric at [" << row << "][" << col
<< "]" << endl;
        }
    }

    mapName = name;
    fin.close();
    return true;
}

bool CollegeMap::isLoaded() const
{
    return nodesTotal > 0;
}

string CollegeMap::getName() const
{
```

```
    return mapName;
}

int CollegeMap::getNodesTotal() const
{
    return nodesTotal;
}

bool CollegeMap::isValidNode(int node) const
{
    return node >= 0 && node < nodesTotal;
}

// -1 if out of range
int CollegeMap::getWeight(int from, int to) const
{
    if (!isValidNode(from) || !isValidNode(to))
        return -1;
    return weights[from][to];
}

// sets both directions
bool CollegeMap::setWeight(int from, int to, int weight)
{
    if (!isValidNode(from) || !isValidNode(to))
    {
        cout << "setWeight: node index out of range" << endl;
        return false;
    }
    if (weight < 0)
    {
        cout << "setWeight: weight cannot be negative" << endl;
        return false;
    }
    if (from == to && weight != 0)
    {
        cout << "setWeight: a node's weight to itself must be 0" << endl;
        return false;
    }

    weights[from][to] = weight;
    weights[to][from] = weight;
    return true;
}
```

A.3 Dijkstra.h

```
#ifndef DIJKSTRA_H
#define DIJKSTRA_H

#include <iostream>
#include <string>
#include "CollegeMap.h"

class Dijkstra
{
    CollegeMap* pMap;   // not owned, never deleted
    int nodes;
    int source;         // -1 = not run yet
    int* dist;
    bool* processed;
    int* parent;

    void allocate(int size);
    void release();
    int minDistance() const;
    void printPath(std::ostream& stream, int j) const;

public:
    Dijkstra(CollegeMap* aMap);
    ~Dijkstra();

    // no copying allowed
    Dijkstra(const Dijkstra& ob) = delete;
    Dijkstra& operator=(const Dijkstra& ob) = delete;

    bool run(int src);
    int getSource() const;
    bool isReachable(int dest) const;
    int getDistance(int dest) const;
    int getRoute(int dest, int* route) const;

    void printRoute(int dest) const;
    bool saveResults(std::string fileName) const;

    friend std::ostream& operator<<(std::ostream& stream, const Dijkstra& ob);
};

#endif
```

A.4 Dijkstra.cpp

```
#include "Dijkstra.h"
#include <fstream>
#include <iomanip>
#include <cstdlib>
using namespace std;

// means not reached yet
const int INFINITE_DIST = 2147483647;

// store map pointer only
Dijkstra::Dijkstra(CollegeMap* aMap)
{
    pMap = aMap;
    nodes = 0;
    source = -1;
    dist = 0;
    processed = 0;
    parent = 0;
}

// frees arrays, not map
Dijkstra::~Dijkstra()
{
    release();
}

void Dijkstra::allocate(int size)
{
    nodes = size;

    dist = new int[nodes];
    processed = new bool[nodes];
    parent = new int[nodes];

    if (!dist || !processed || !parent)
    {
        cout << "Allocation failed." << endl;
        exit(1);
    }
}

void Dijkstra::release()
{
    if (dist)
        delete[] dist;
    if (processed)
        delete[] processed;
    if (parent)
        delete[] parent;
    dist = 0;
    processed = 0;
    parent = 0;
    nodes = 0;
}

// closest unprocessed node, -1 if none
int Dijkstra::minDistance() const
{
    int min = INFINITE_DIST;
    int minIndex = -1;

    for (int v = 0; v < nodes; v++)
    {
        if (!processed[v] && dist[v] < min)
```

```
        {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

bool Dijkstra::run(int src)
{
    if (!pMap || !pMap->isLoaded())
    {
        cout << "Dijkstra: no map loaded" << endl;
        source = -1;
        return false;
    }
    if (!pMap->isValidNode(src))
    {
        cout << "Dijkstra: invalid source node " << src << endl;
        source = -1;
        return false;
    }

    // map size may change
    release();
    allocate(pMap->getNodesTotal());

    for (int i = 0; i < nodes; i++)
    {
        dist[i] = INFINITE_DIST;
        processed[i] = false;
        parent[i] = -1;
    }
    dist[src] = 0;

    // stop when none reachable
    int u = minDistance();
    while (u != -1)
    {
        processed[u] = true;

        for (int v = 0; v < nodes; v++)
        {
            int w = pMap->getWeight(u, v);
            if (!processed[v] && w > 0 && dist[u] + w < dist[v])
            {
                parent[v] = u;
                dist[v] = dist[u] + w;
            }
        }

        u = minDistance();
    }

    source = src;
    return true;
}

int Dijkstra::getSource() const
{
    return source;
}

bool Dijkstra::isReachable(int dest) const
{
```

```
    if (source == -1 || dest < 0 || dest >= nodes)
        return false;
    return dist[dest] != INFINITE_DIST;
}

// -1 if unreachable
int Dijkstra::getDistance(int dest) const
{
    if (!isReachable(dest))
        return -1;
    return dist[dest];
}

// fills route, returns length
int Dijkstra::getRoute(int dest, int* route) const
{
    if (!isReachable(dest))
        return 0;

    // count back to source
    int count = 0;
    int node = dest;
    while (node != -1)
    {
        count++;
        node = parent[node];
    }

    // fill in backwards
    int i = count - 1;
    node = dest;
    while (node != -1)
    {
        route[i] = node;
        i--;
        node = parent[node];
    }
    return count;
}

// print path from source to destination
void Dijkstra::printPath(ostream& stream, int j) const
{
    // base case: j is source
    if (parent[j] == -1)
    {
        stream << j;
        return;
    }
    printPath(stream, parent[j]);
    stream << " -> " << j;
}

// one route to screen
void Dijkstra::printRoute(int dest) const
{
    if (source == -1)
    {
        cout << "No results - call run() first" << endl;
        return;
    }
    if (dest < 0 || dest >= nodes)
    {
        cout << "Invalid destination " << dest << endl;
        return;
```

```
    }
    if (!isReachable(dest))
    {
        cout << "Node " << dest << " is unreachable from node " << source << endl;
        return;
    }

    cout << "Node " << source << " to " << dest << ", distance " << dist[dest] <<
": ";
    printPath(cout, dest);
    cout << endl;
}

// all routes, any stream
ostream& operator<<(ostream& stream, const Dijkstra& ob)
{
    if (ob.source == -1)
    {
        stream << "No results - call run() first" << endl;
        return stream;
    }

    stream << "Shortest routes from node " << ob.source << endl;
    stream << "Destination  Distance    Route" << endl;

    for (int dest = 0; dest < ob.nodes; dest++)
    {
        if (dest != ob.source)
        {
            stream << setw(11) << dest;
            if (!ob.isReachable(dest))
            {
                stream << setw(10) << "-" << "    unreachable" << endl;
            }
            else
            {
                stream << setw(10) << ob.dist[dest] << "    ";
                ob.printPath(stream, dest);
                stream << endl;
            }
        }
    }

    return stream;
}

// map and routes to file
bool Dijkstra::saveResults(string fileName) const
{
    if (source == -1)
    {
        cout << "Nothing to save - call run() first" << endl;
        return false;
    }

    ofstream fout(fileName);
    if (!fout)
    {
        cout << "Cannot open output file: " << fileName << endl;
        return false;
    }

    fout << *pMap << endl;
    fout << *this;
    fout.close();
```

```
    return true;
}
```

A.5 main.cpp (MapDemo)

```
#include <iostream>
#include <string>
#include <cstdlib>
#include "CollegeMap.h"
#include "Dijkstra.h"
using namespace std;

int main()
{
    // load and show map
    CollegeMap mtu("MTU Map.txt");
    if (!mtu.isLoaded())
    {
        cout << "Could not load map" << endl;
        return 1;
    }
    cout << mtu << endl;

    // atoi gives 0 if bad
    string input = "";
    cout << "Enter source node (0-" << mtu.getNodesTotal() - 1 << "): ";
    cin >> input;
    int src = atoi(input.c_str());
    if (src == 0 && !(input == "0"))
        cout << "Not a number - using node 0" << endl;
    cout << endl;

    // new so we can delete
    Dijkstra* planner = new Dijkstra(&mtu);
    if (planner->run(src))
    {
        cout << *planner;
        if (planner->saveResults("Routes.txt"))
            cout << "Results saved to Routes.txt" << endl;
    }
    delete planner;     // map survives (aggregation)

    cout << endl << "Dijkstra object destroyed - map still usable: "
        << mtu.getName() << ", " << mtu.getNodesTotal() << " nodes" << endl <<
endl;

    // change weight and rerun
    Dijkstra planner2(&mtu);
    planner2.run(0);
    cout << "Before change: ";
    planner2.printRoute(4);

    mtu.setWeight(0, 4, 3);
    planner2.run(0);
    cout << "After adding edge 0-4: ";
    planner2.printRoute(4);
    cout << endl;

    // copy and assign
    CollegeMap copyMap = mtu;
    CollegeMap assigned;
    assigned = mtu;
```

```
    copyMap.setWeight(0, 1, 99);
    cout << "After changing copyMap(0,1) to 99:" << endl;
    cout << "  copyMap(0,1)  = " << copyMap.getWeight(0, 1) << endl;
    cout << "  mtu(0,1)      = " << mtu.getWeight(0, 1) << endl;
    cout << "  assigned(0,1) = " << assigned.getWeight(0, 1) << endl;

    if (copyMap == mtu)
        cout << "  copyMap == mtu ? yes" << endl;
    else
        cout << "  copyMap == mtu ? no" << endl;

    if (assigned == mtu)
        cout << "  assigned == mtu ? yes" << endl;
    else
        cout << "  assigned == mtu ? no" << endl;

    assigned = assigned;    // self assignment
    if (assigned == mtu)
        cout << "  after self-assignment, assigned == mtu ? yes" << endl;
    else
        cout << "  after self-assignment, assigned == mtu ? no" << endl;

    return 0;
}
```

A.6 test.cpp (MapTests)

```
#include "pch.h"
#include <cstdio>
#include <fstream>
#include <string>
#include "CollegeMap.h"
#include "Dijkstra.h"
using namespace std;

// write a test file
void writeFile(string fileName, string contents)
{
    ofstream fout(fileName);
    fout << contents;
    fout.close();
}

// read file as words
string readWords(string fileName)
{
    ifstream fin(fileName);
    string word, text;
    while (fin >> word)
    {
        if (text.length() > 0)
            text += " ";
        text += word;
    }
    return text;
}

// 3 node map, Appendix 4
class TinyMapTest : public ::testing::Test
{
protected:
    void SetUp() override
    {
        writeFile("tiny.txt", "Tiny_Map\n3\n0 5 0\n5 0 2\n0 2 0\n");
    }

    void TearDown() override
    {
        remove("tiny.txt");
        remove("out.txt");
    }
};

// MTU map, Appendix 3
class MtuMapTest : public ::testing::Test
{
protected:
    void SetUp() override
    {
        string contents = "MTU_Map\n9\n";
        contents += "0 4 0 0 0 0 0 8 0\n";
        contents += "4 0 8 0 0 0 0 11 0\n";
        contents += "0 8 0 7 0 4 0 0 2\n";
        contents += "0 0 7 0 9 14 0 0 0\n";
        contents += "0 0 0 9 0 10 0 0 0\n";
        contents += "0 0 4 0 10 0 2 0 0\n";
        contents += "0 0 0 14 0 2 0 1 6\n";
        contents += "8 11 0 0 0 0 1 0 7\n";
        contents += "0 0 2 0 0 0 6 7 0\n";
        writeFile("mtu.txt", contents);
    }
```

```
    void TearDown() override
    {
        remove("mtu.txt");
    }
};

// two halves, some unreachable
class SplitMapTest : public ::testing::Test
{
protected:
    void SetUp() override
    {
        writeFile("split.txt", "Split_Map\n4\n0 3 0 0\n3 0 0 0\n0 0 0 4\n0 0 4
0\n");
    }

    void TearDown() override
    {
        remove("split.txt");
    }
};

// ---------- CollegeMap ----------

// CM1
TEST(CollegeMapTest, EmptyMapIsSafeToUse)
{
    CollegeMap empty;
    EXPECT_FALSE(empty.isLoaded());
    EXPECT_EQ(0, empty.getNodesTotal());
    EXPECT_EQ("None", empty.getName());

    CollegeMap copy = empty;
    CollegeMap assigned;
    assigned = empty;
    EXPECT_FALSE(copy.isLoaded());
    EXPECT_FALSE(assigned.isLoaded());

    ofstream fout("empty.txt");
    fout << empty;
    fout.close();
    EXPECT_EQ("Map: (no map loaded)", readWords("empty.txt"));
    remove("empty.txt");
}

// CM2
TEST(CollegeMapTest, MissingAndMalformedFilesFailToLoad)
{
    CollegeMap map;
    EXPECT_FALSE(map.load("missing.txt"));

    writeFile("bad.txt", "Bad_Map\nthree\n0 5\n5 0\n");     // non-numeric count
    EXPECT_FALSE(map.load("bad.txt"));

    writeFile("bad.txt", "Bad_Map\n3\n0 5 0\n5 0 2\n0 2\n"); // too few values
    EXPECT_FALSE(map.load("bad.txt"));

    writeFile("bad.txt", "Bad_Map\n2\n0 -5\n-5 0\n");        // negative weight
    EXPECT_FALSE(map.load("bad.txt"));

    writeFile("bad.txt", "Bad_Map\n2\n1 5\n5 0\n");          // non-zero diagonal
    EXPECT_FALSE(map.load("bad.txt"));

    EXPECT_FALSE(map.isLoaded());   // left empty
```

```
    remove("bad.txt");
}

// CM3, CM4
TEST_F(TinyMapTest, LoadsNameNodesAndWeights)
{
    CollegeMap map("tiny.txt");
    EXPECT_TRUE(map.isLoaded());
    EXPECT_EQ("Tiny_Map", map.getName());
    EXPECT_EQ(3, map.getNodesTotal());
    EXPECT_EQ(5, map.getWeight(0, 1));
    EXPECT_EQ(2, map.getWeight(2, 1));
    EXPECT_EQ(0, map.getWeight(0, 2));
}

// CM5
TEST_F(TinyMapTest, InvalidIndicesAndWeightsRejected)
{
    CollegeMap map("tiny.txt");
    EXPECT_TRUE(map.isValidNode(2));
    EXPECT_FALSE(map.isValidNode(3));
    EXPECT_FALSE(map.isValidNode(-1));
    EXPECT_EQ(-1, map.getWeight(0, 3));

    EXPECT_FALSE(map.setWeight(3, 0, 1));    // no node 3
    EXPECT_FALSE(map.setWeight(0, 1, -4));   // negative
    EXPECT_FALSE(map.setWeight(1, 1, 3));    // same node
    EXPECT_EQ(5, map.getWeight(0, 1));       // unchanged
}

// CM5
TEST_F(TinyMapTest, SetWeightChangesBothDirections)
{
    CollegeMap map("tiny.txt");
    EXPECT_TRUE(map.setWeight(0, 2, 9));
    EXPECT_EQ(9, map.getWeight(0, 2));
    EXPECT_EQ(9, map.getWeight(2, 0));
}

// CM4, CM6
TEST_F(TinyMapTest, DisplayShowsNameNodesAndMatrix)
{
    CollegeMap map("tiny.txt");
    ofstream fout("out.txt");
    fout << map;
    fout.close();
    EXPECT_EQ("Map: Tiny_Map Nodes: 3 0 1 2 0 0 5 0 1 5 0 2 2 0 2 0",
readWords("out.txt"));
}

// CM7
TEST_F(TinyMapTest, CopyIsIndependent)
{
    CollegeMap original("tiny.txt");
    CollegeMap copy = original;
    EXPECT_TRUE(copy == original);

    copy.setWeight(0, 1, 99);
    EXPECT_EQ(5, original.getWeight(0, 1));
    EXPECT_FALSE(copy == original);
}

// CM7
TEST_F(TinyMapTest, AssignmentIsIndependent)
{
```

```
    CollegeMap original("tiny.txt");
    CollegeMap assigned;
    assigned = original;
    EXPECT_TRUE(assigned == original);

    original.setWeight(1, 2, 50);
    EXPECT_EQ(2, assigned.getWeight(1, 2));
}

// CM7
TEST_F(TinyMapTest, SelfAssignmentKeepsData)
{
    CollegeMap map("tiny.txt");
    map = map;
    EXPECT_TRUE(map.isLoaded());
    EXPECT_EQ(5, map.getWeight(0, 1));
}

// CM9
TEST_F(TinyMapTest, ReloadReplacesOldMap)
{
    CollegeMap map("tiny.txt");
    writeFile("two.txt", "Two_Map\n2\n0 1\n1 0\n");
    EXPECT_TRUE(map.load("two.txt"));
    EXPECT_EQ(2, map.getNodesTotal());
    EXPECT_EQ("Two_Map", map.getName());
    remove("two.txt");
}

// ---------- Dijkstra ----------

// D3
TEST(DijkstraTest, OneNodeMapRuns)
{
    writeFile("one.txt", "One_Map\n1\n0\n");
    CollegeMap map("one.txt");
    Dijkstra planner(&map);
    EXPECT_TRUE(planner.run(0));
    EXPECT_EQ(0, planner.getDistance(0));
    remove("one.txt");
}

// D3, CM8
TEST_F(MtuMapTest, DistancesFromNodeZeroMatchBrief)
{
    CollegeMap map("mtu.txt");
    Dijkstra planner(&map);
    ASSERT_TRUE(planner.run(0));
    EXPECT_EQ(4, planner.getDistance(1));
    EXPECT_EQ(12, planner.getDistance(2));
    EXPECT_EQ(19, planner.getDistance(3));
    EXPECT_EQ(21, planner.getDistance(4));
    EXPECT_EQ(11, planner.getDistance(5));
    EXPECT_EQ(9, planner.getDistance(6));
    EXPECT_EQ(8, planner.getDistance(7));
    EXPECT_EQ(14, planner.getDistance(8));
}

// D3: rerun, new source
TEST_F(MtuMapTest, RerunFromNodeThree)
{
    CollegeMap map("mtu.txt");
    Dijkstra planner(&map);
    planner.run(0);
    planner.run(3);
```

```
    EXPECT_EQ(3, planner.getSource());
    EXPECT_EQ(19, planner.getDistance(0));
    EXPECT_EQ(15, planner.getDistance(1));
    EXPECT_EQ(7, planner.getDistance(2));
    EXPECT_EQ(0, planner.getDistance(3));
    EXPECT_EQ(9, planner.getDistance(4));
    EXPECT_EQ(11, planner.getDistance(5));
    EXPECT_EQ(13, planner.getDistance(6));
    EXPECT_EQ(14, planner.getDistance(7));
    EXPECT_EQ(9, planner.getDistance(8));
}

// D4, bug 1: wrong source parent
TEST_F(MtuMapTest, RouteFromNodeThreeToSeven)
{
    CollegeMap map("mtu.txt");
    Dijkstra planner(&map);
    planner.run(3);
    int route[9];
    ASSERT_EQ(5, planner.getRoute(7, route));
    EXPECT_EQ(3, route[0]);
    EXPECT_EQ(2, route[1]);
    EXPECT_EQ(5, route[2]);
    EXPECT_EQ(6, route[3]);
    EXPECT_EQ(7, route[4]);

    planner.printRoute(7);      // valid
    planner.printRoute(9);      // invalid
    EXPECT_EQ(14, planner.getDistance(7));
}

// D3
TEST_F(TinyMapTest, InvalidSourceAndEmptyMapRejected)
{
    CollegeMap map("tiny.txt");
    Dijkstra planner(&map);
    EXPECT_FALSE(planner.run(-1));
    EXPECT_FALSE(planner.run(3));
    EXPECT_EQ(-1, planner.getSource());

    CollegeMap empty;
    Dijkstra planner2(&empty);
    EXPECT_FALSE(planner2.run(0));
}

// D4, bug 2: unreachable went negative
TEST_F(SplitMapTest, UnreachableNodeReported)
{
    CollegeMap map("split.txt");
    Dijkstra planner(&map);
    planner.run(0);
    EXPECT_TRUE(planner.isReachable(1));
    EXPECT_EQ(3, planner.getDistance(1));
    EXPECT_FALSE(planner.isReachable(3));
    EXPECT_EQ(-1, planner.getDistance(2));
    EXPECT_EQ(-1, planner.getDistance(3));

    int route[4];
    EXPECT_EQ(0, planner.getRoute(3, route));

    planner.run(2);             // other half works
    EXPECT_EQ(4, planner.getDistance(3));
}

// D4, bug 3: source printed as 0
```

```
TEST_F(TinyMapTest, PrintAllShowsRealSource)
{
    CollegeMap map("tiny.txt");
    Dijkstra planner(&map);
    planner.run(2);
    ofstream fout("out.txt");
    fout << planner;
    fout.close();
    EXPECT_EQ("Shortest routes from node 2 Destination Distance Route 0 7 2 -> 1 ->
0 1 2 2 -> 1",
        readWords("out.txt"));
}

// D5
TEST_F(TinyMapTest, SaveResultsWritesMapAndRoutes)
{
    CollegeMap map("tiny.txt");
    Dijkstra planner(&map);
    planner.run(0);
    EXPECT_TRUE(planner.saveResults("out.txt"));
    string expected = "Map: Tiny_Map Nodes: 3 0 1 2 0 0 5 0 1 5 0 2 2 0 2 0 ";
    expected += "Shortest routes from node 0 Destination Distance Route 1 5 0 -> 1
2 7 0 -> 1 -> 2";
    EXPECT_EQ(expected, readWords("out.txt"));
}

// D1, D2: sees map changes
TEST_F(TinyMapTest, DijkstraSeesMapChanges)
{
    CollegeMap map("tiny.txt");
    Dijkstra planner(&map);
    planner.run(0);
    EXPECT_EQ(7, planner.getDistance(2));

    map.setWeight(0, 2, 1);     // direct 0-2 edge
    planner.run(0);
    EXPECT_EQ(1, planner.getDistance(2));

    writeFile("big.txt", "Big_Map\n4\n0 1 0 0\n1 0 1 0\n0 1 0 1\n0 0 1 0\n");
    map.load("big.txt");        // bigger map
    EXPECT_TRUE(planner.run(0));
    EXPECT_EQ(3, planner.getDistance(3));
    remove("big.txt");
}

// D1, D7
TEST_F(TinyMapTest, MapSurvivesDijkstraDestruction)
{
    CollegeMap map("tiny.txt");
    Dijkstra* planner = new Dijkstra(&map);
    planner->run(0);
    delete planner;     // map untouched
    EXPECT_TRUE(map.isLoaded());
    EXPECT_EQ(5, map.getWeight(0, 1));
}
```

Appendix B: AI Transcript

Every prompt and response, in order. The first prompt is the brief from “Assignment Overview” to “Sample Output” with Appendices 1 and 3, unchanged. The AI also returned DijkstraSolution.zip. Its code, set up as a Visual Studio solution with the ported tests, is in the AI solution zip submitted with this report.

B.1 Prompt 1

Assignment Overview:

The complete system consists of two classes:

- CollegeMap loads the map metadata (map name and number of nodes) and the edge weights from a text file such as ‘MTU Map.txt’ (Appendix 3). The size of the map is not known until the file is read, so the weights must be stored in dynamically allocated memory. The user of the class must be able to inspect the metadata and read and modify individual edge weights.

- Dijkstra uses the data held by a CollegeMap object. Given a start node, it executes the Dijkstra algorithm and can then report the shortest distance and route from the start node to any other node, i.e. the route from the current location to the destination.

Fixed Constraints:

The following constraints are not open to interpretation. A solution that breaks one of them will be capped in the relevant section of the marking scheme, however well it works otherwise.

1. The solution should present each class declared in its own header (.h) file and defined in its own source (.cpp) file.

2. Inheritance must not be used in your classes.

3. The edge-weight matrix and the working arrays used by the algorithm (distances, the processed-node set and the parent array) must be allocated with new[] and released with delete[]. std::vector, std::array, smart pointers and other STL containers must not be used for this data. std::string and the file stream classes may be used.

4. The relationship between Dijkstra and CollegeMap must be aggregation: a CollegeMap must be able to outlive any Dijkstra object that uses it and destroying a Dijkstra object must never destroy or release the data belonging to a CollegeMap.

5. Every allocation must be released exactly once, on every path through the program, including the paths where a file fails to load.

6. The code must build without warnings at Visual Studio warning level 3 (/W3).

CollegeMap Class – Required Behaviour

The requirements below describe the behaviour the class must provide. They are not a list of methods: one behaviour may need several methods and one method may satisfy several behaviours. The identifiers (CM1, CM2, …) will be used again in Part C, where your tests must be traced back to them.

CM1 Construction. It must be possible to create a CollegeMap with no map loaded and to create one from a file name. An object with no map loaded, or whose file could not be read, must still be safe to display, copy, assign and destroy.

CM2 Loading and validation. The class must confirm that the file exists and that its contents are usable before relying on them. You must decide what "usable" means. Consider, for example, a missing or non-numeric node count, too few rows or values, negative weights, a non-zero diagonal, or a matrix that is not symmetric. You decide which of these are errors, how the class reports a problem to the code using it and what state the object is left in afterwards. See Appendix 2 for sample file-access code.

CM3 Storage. The weights must be held in a two-dimensional structure allocated at run time to fit the number of nodes. One approach is a handle to an array of row pointers, each pointing to a separately allocated row, as shown below. Another is a single contiguous block of NodesTotal × NodesTotal values indexed as row * NodesTotal + col. Choose one and justify the choice in Part B.

```
WeightsArray = new int*[NodesTotal];
for (int row = 0; row < NodesTotal; ++row)
   WeightsArray[row] = new int[NodesTotal];
```

CM4 Metadata. The map name and the number of nodes must be available to code using the class and must be displayable.

CM5 Weight access and modification. Code using the class must be able to read and change an individual edge weight. Decide what should happen when an index is out of range or a weight is negative. Decide also whether the map is undirected: should changing the weight from node A to node B also change the weight from B to A? Aim for an interface that makes misuse difficult.

CM6 Display. The metadata and the weight matrix must be displayable in a readable form. Whether this is done through member functions, an overloaded stream insertion operator, or both, is your decision.

CM7 Copying. It must be possible to create a CollegeMap as a copy of another (initialisation) and to assign one CollegeMap to another (assignment). In both cases the result must be a deep, independent copy: changing the copy must not change the original. Previous versions of this assignment gave you the signature of the assignment operator; this version does not. Think carefully about what its parameter type and return type should be, what must happen when an object is assigned to itself and how to avoid writing the same copying logic twice.

CM8 Supplying weights to other objects. Another object (in particular, a Dijkstra object) must be able to obtain the edge weights. You might expose the internal storage, return an independent copy, or provide element access only. Each option has consequences for encapsulation, efficiency and, crucially, for who is responsible for releasing memory. You must discuss this choice in Part B.

CM9 Release. All dynamically allocated memory must be released when the object is destroyed and before existing storage is replaced (for example, on assignment or reloading).

Operator Overloading

In addition to the assignment operator, your CollegeMap class must provide at least two further overloaded operators. Use operators whose meaning would be guessed correctly without reading your documentation i.e. the operator's use should be intuitive. Equality comparison, element access and stream insertion are all worth considering, but the choice is yours and you are free to choose others. An operator that would surprise its user (for example, using + to change a weight) will lose marks however cleverly it is implemented.

Hint: think about which operators would make the unit tests you write in Part C easier to write and easier to read.

Dijkstra Class – Required Behaviour

D1 Association. A Dijkstra object is created for an existing CollegeMap. You decide how it refers to that map (by copy, by reference or by pointer), subject to Fixed Constraint 4. Note that the previous version of this assignment stored a copy of the map inside the Dijkstra object. Was that aggregation? Discuss this in Part B.

D2 Working storage. The distance, processed node and parent arrays must be allocated dynamically to match the size of the map. Decide when they are allocated (on construction, or on each run) and what should happen if the map is modified after the Dijkstra object was created.

D3 Execution. The algorithm must run from any valid source node and must give correct results when run repeatedly on the same object with different source nodes. An invalid source node must be handled sensibly.

D4 Results. After a run, it must be possible to obtain the shortest distance and route from the source to (a) a single chosen destination and (b) every other node. A node that cannot be reached must be reported in a meaningful way, not as 2147483647.

D5 Saving results. The complete set of results must be writable to a text file whose name is supplied by the code using the class.

D6 Copying. Decide whether Dijkstra objects may be copied and assigned. If they may, implement copying correctly. If they may not, prevent it (research how C++ allows you to do this). Either way, justify the decision in Part B.

D7 Release. All memory allocated by the Dijkstra object must be released when it is destroyed, without affecting the CollegeMap it used.

The Reference Algorithm

The C code in Appendix 1 is your starting point. It is written for a fixed-size global array and a source node of 0. It also contains at least two defects that go unnoticed when the sample map is used with node 0 as the source. Part of the task is to find them, correct them in your version and write a unit test in Part C that would have caught each one.

Do not simply transliterate the C functions into methods. Decide which of them become public methods, which become private helpers and which should disappear altogether now that the data they need is held as member data.

Demonstration Program:

No main() is supplied. Write a short console program that demonstrates, at a minimum:

- loading ‘MTU Map.txt’ and displaying its metadata and weights;
- changing an edge weight and showing the effect on a route;
- copying and assigning a map and showing that the copies are independent;
- computing routes from a source node chosen by the user, displaying them and saving them to a file;
- destroying a Dijkstra object, while the CollegeMap used remains valid and usable, demonstrating aggregation.

Keep the demonstration program in a separate project from the classes so that the classes can be tested independently (see Appendix 4).

Sample Output:

The layout below is indicative only; the format of your output is your decision. It shows the results for the map in Appendix 3 with node 0 as the source.

Map: MTU_Map Nodes: 9
Shortest routes from node 0

Destination	Distance	Route
1	4	0 -> 1
2	12	0 -> 1 -> 2
3	19	0 -> 1 -> 2 -> 3
4	21	0 -> 7 -> 6 -> 5 -> 4
5	11	0 -> 7 -> 6 -> 5
6	9	0 -> 7 -> 6
7	8	0 -> 7
8	14	0 -> 1 -> 2 -> 8

Appendix 1: Reference Dijkstra C Code

Source: https://www.geeksforgeeks.org/printing-paths-dijkstras-shortest-path-algorithm/

```
#include "pch.h"
#include <iostream>
using namespace std;

// Number of vertices
// in the graph
constexpr auto V = 9;

void dijkstra(int graph[V][V], int src);
int minDistance(int dist[], bool sptSet[]);
void printPath(int parent[], int j);
int printSolution(int dist[], int n, int parent[]);

int main()
{
   // Let us create the example
   // graph discussed above
   int graph[V][V] = { {0, 4, 0, 0, 0, 0, 8, 0},
                       {4, 0, 8, 0, 0, 0, 0, 11, 0},
                       {0, 8, 0, 7, 0, 4, 0, 0, 2},
                       {0, 0, 7, 0, 9, 14, 0, 0, 0},
                       {0, 0, 0, 9, 0, 10, 0, 0, 0},
                       {0, 0, 4, 0, 10, 0, 2, 0, 0},
                       {0, 0, 0, 14, 0, 2, 0, 1, 6},
                       {8, 11, 0, 0, 0, 0, 1, 0, 7},
                       {0, 0, 2, 0, 0, 0, 6, 7, 0}
   };
```

```
    dijkstra(graph, 0);
    return 0;
}

// A utility function to find the
// vertex with minimum distance
// value, from the set of vertices
// not yet included in shortest
// path tree
int minDistance(int dist[], bool sptSet[])
{
    // Initialize min value
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++)
        if (sptSet[v] == false &&
            dist[v] <= min)
            min = dist[v], min_index = v;

    return min_index;
}

// Function to print shortest
// path from source to j
// using parent array
void printPath(int parent[], int j)
{
    // Base Case : If j is source
    if (parent[j] == -1)
        return;

    printPath(parent, parent[j]);

    printf("%d ", j);
}

// A utility function to print
// the constructed distance
// array
int printSolution(int dist[], int n, int parent[])
{
    int src = 0;
    printf("Vertex\t Distance\tPath");
    for (int i = 1; i < V; i++)
    {
        printf("\n%d -> %d \t\t %d\t\t%d ",
            src, i, dist[i], src);
        printPath(parent, i);
```

```
    }
    return 0;
}

// Function that implements Dijkstra's
// single source shortest path
// algorithm for a graph represented
// using adjacency matrix representation
void dijkstra(int graph[V][V], int src)
{
    // The output array. dist[i] will hold the shortest distance from src to i
    int dist[V];

    // sptSet[i] will true if vertex i is included / in shortest
    // path tree or shortest distance from src to i is finalized
    bool sptSet[V];

    // Parent array to store shortest path tree
    int parent[V];

    // Initialize all distances as INFINITE and stpSet[] as false
    for (int i = 0; i < V; i++)
    {
        parent[0] = -1;
        dist[i] = INT_MAX;
        sptSet[i] = false;
    }

    // Distance of source vertex from itself is always 0
    dist[src] = 0;

    // Find shortest path
    // for all vertices
    for (int count = 0; count < V - 1; count++)
    {
        // Pick the minimum distance vertex from the set of vertices not yet processed.
        // u is always equal to src in first iteration.
        int u = minDistance(dist, sptSet);

        // Mark the picked vertex
        // as processed
        sptSet[u] = true;

        // Update dist value of the adjacent vertices of the picked vertex.
        for (int v = 0; v < V; v++)

            // Update dist[v] only if is not in sptSet, there is
            // an edge from u to v and total weight of path from
```

```
        // src to v through u is smaller than current value of dist[v]
        if (!sptSet[v] && graph[u][v] &&
            dist[u] + graph[u][v] < dist[v])
        {
            parent[v] = u;
            dist[v] = dist[u] + graph[u][v];
        }
    }

    // print the constructed
    // distance array
    printSolution(dist, V, parent);
}
```

Appendix 3: Sample College Map File: 'MTU Map.txt'

```
MTU_Map
9
0  4  0  0  0  0  0  8  0
4  0  8  0  0  0  0  11 0
0  8  0  7  0  4  0  0  2
0  0  7  0  9  14 0  0  0
0  0  0  9  0  10 0  0  0
0  0  4  0  10 0  2  0  0
0  0  0  14 0  2  0  1  6
8  11 0  0  0  0  1  0  7
0  0  2  0  0  0  6  7  0
```

A weight of 0 indicates that there is no direct edge between two nodes.

B.2 Response 1

Got the brief. Before anything else, here are the defects in the Appendix 1 code, since finding them is part of the marks:

1. `parent[0] = -1` instead of `parent[src] = -1` (in the init loop)

With source 0 this happens to be right. With any other source, `parent[src]` is never set because the source is never relaxed, so it's left as garbage. `printPath` then recurses into an uninitialised index: undefined behaviour, usually a crash or garbage route.

Test that catches it: source 3, destination 4, expect route `3 -> 4`, distance 9.

2. Integer overflow on unreachable nodes (`minDistance` uses `<=` against `INT_MAX`)

Once only unreachable nodes are left, `<=` still picks one with `dist == INT_MAX`. Then `dist[u] + graph[u][v]` overflows to a negative number, which is less than `INT_MAX`, so a node that can't be reached gets a negative distance and a bogus parent. If you switch to `<` without other changes, `min_index` is returned uninitialised instead.

Fix: use `<`, return -1 when nothing is found and break out of the loop and also guard the relaxation with `dist[u] != INT_MAX`.

Test that catches it: a 4-node map where 0–1 are connected, 2–3 are connected and the two pairs aren't linked. From source 0, node 3 must be reported unreachable, not given a negative distance.

Secondary issues (not the "hidden" defects, but you'll fix them anyway):

* `printSolution` hardcodes `src = 0` and ignores `n`.

* Unreachable nodes print as 2147483647 (D4 says not to do this).

* `min_index` is uninitialised in all cases.

What do you want from me on this?

* Full build: both classes, the demo project and the Google Test project.

* Design first: a decision list for CM3, CM5, CM7, CM8, D1, D2 and D6 that you can write Part B from, then the code.

* Review your attempt: if you've already started, send it over and I'll check it against the constraints.

B.3 Prompt 2 (follow-up 1)

Full build: both classes, the demo project and the Google Test project.

B.4 Response 2

Building it now: two classes in a static library, a demo console app and a Google Test project. I'll compile and run everything here before handing it over.

The full build is in `DijkstraSolution.zip`. It contains the class library, the demo project, the Google Test project and a README covering Visual Studio setup, Part B design notes and a Part C traceability table. All 65 tests pass, it compiles with no warnings under strict GCC flags, the demo's node-0 output matches the brief's sample table and the memory checkers found no leaks or errors.

The supplied `MTU Map.txt` isn't symmetric. Row 3 has a 14 at column 5 but row 5 has 0 there and row 6 has a 14 at column 3 that row 3 doesn't match. I treat that as a load error. The demo shows the supplied file being rejected with the reason, then loads `MTU Map (corrected).txt`, which uses the original graph's single 3–5 edge. Results from node 0 are the same either way. If you'd rather accept asymmetric files, that's a design change worth deciding before you write Part B.

Both defects are confirmed against the original C code. Source 3 segfaults and a disconnected map gives a node a distance of −2147483646. I put each defect back into my version to check the tests: defect 1 crashes its test while the node-0 sample test still passes and defect 2 fails both unreachable-node tests.

I couldn't build it with MSVC here. Build it once in VS 2022 at /W3 and check the Output window before submitting. The README has the project setup steps. If the Google Test template uses precompiled headers, keep `#include "pch.h"` as the first line of `test.cpp`.